■# Veeva Today's Questions Only

This sheet contains only the questions shared today: Java MCQs, one SQL query, and latest coding/interview problems.

Java MCQs - Constructor, Inner Class, Thread, Inheritance

1. Private Constructor And Private Field

What is the output?

```
class Helper {
    private int data;
    private Helper() {
        data = 5;
    }
}
public class Test {
    public static void main(String[] args) {
        Helper help = new Helper();
        System.out.println(help.data);
    }
}
```

A. Compilation error

B. 5

C. Runtime error

D. None of these

Answer: A

Explanation: `Helper()` is private and `data` is private, so both cannot be accessed from `Test`.

2. Constructor Inside try Block

What is the output?

```
public class Test implements Runnable {
    public void run() {
        System.out.printf(" Thread's running ");
    }

    try {
        public Test() {
            Thread.sleep(5000);
        }
    } catch (InterruptedException e) {
        e.printStackTrace();
    }

    public static void main(String[] args) {
        Test obj = new Test();
        Thread thread = new Thread(obj);
        thread.start();
        System.out.printf(" GFG ");
    }
}
```

A. GFG Thread's running

B. Thread's running GFG

C. Compilation error

D. Runtime error

Answer: C

Explanation: A constructor cannot be declared inside a `try` block.

3. Private Constructor With Static Factory

What is the output?

```
class Temp {
    private Temp(int data) {
        System.out.printf(" Constructor called ");
    }

    protected static Temp create(int data) {
        Temp obj = new Temp(data);
        return obj;
    }
```

```
    public void myMethod() {
        System.out.printf(" Method called ");
    }
}

public class Test {
    public static void main(String[] args) {
        Temp obj = Temp.create(20);
        obj.myMethod();
    }
}
```

A. Constructor called Method called

B. Compilation error

C. Runtime error

D. None of the above

Answer: A

Explanation: The private constructor is called from inside the same class using the static factory method `create()`.

4. Constructor Chaining By Object Creation

What is the output?

```
public class Test {
    public Test() {
        System.out.printf("1");
        new Test(10);
        System.out.printf("5");
    }

    public Test(int temp) {
        System.out.printf("2");
        new Test(10, 20);
        System.out.printf("4");
    }

    public Test(int data, int temp) {
        System.out.printf("3");
    }

    public static void main(String[] args) {
        Test obj = new Test();
    }
}
```

A. 12345

B. Compilation error

C. 15

D. Runtime error

Answer: A

Explanation: Flow is `Test()` prints 1, creates `Test(int)` prints 2, creates `Test(int,int)` prints 3, returns to print 4, then returns to print 5.

5. super() Position

What is the output?

```
class Base {
    public static String s = " Super Class ";

    public Base() {
        System.out.printf("1");
    }
}

public class Derived extends Base {
    public Derived() {
        System.out.printf("2");
        super();
    }

    public static void main(String[] args) {
        Derived obj = new Derived();
        System.out.printf(s);
    }
}
```

A. 21 Super Class

B. Super Class 21

C. Compilation error

D. 12 Super Class

Answer: C

Explanation: `super()` must be the first statement in a constructor.

6. Static Nested Class Access

What is the output?

```
public class Outer {
    public static int temp1 = 1;
    private static int temp2 = 2;
    public int temp3 = 3;
    private int temp4 = 4;

    public static class Inner {
        private static int temp5 = 5;

        private static int getSum() {
            return (temp1 + temp2 + temp3 + temp4 + temp5);
        }
    }

    public static void main(String[] args) {
        Outer.Inner obj = new Outer.Inner();
        System.out.println(obj.getSum());
    }
}
```

A. 15

B. 9

C. 5

D. Compilation Error

Answer: D

Explanation: A static nested class cannot directly access non-static variables `temp3` and `temp4`.

7. Local Inner Class

What is the output?

```
public class Outer {
    private static int data = 10;

    private static int LocalClass() {
        class Inner {
            public int data = 20;
            private int getData() {
                return data;
            }
        }
        Inner inner = new Inner();
        return inner.getData();
    }

    public static void main(String[] args) {
        System.out.println(data * LocalClass());
    }
}
```

A. Compilation error

B. Runtime Error

C. 200

D. None of the above

Answer: C

Explanation: Outer static `data` is 10 and local inner class `data` is 20. Result = 10 * 20 = 200.

8. Anonymous Class Reference Type

What is the output?

```
interface Anonymous {
    public int getValue();
}

public class Outer {
    private int data = 15;
```

```
    public static void main(String[] args) {
        Anonymous inner = new Anonymous() {
            int data = 5;

            public int getValue() {
                return data;
            }

            public int getData() {
                return data;
            }
        };

        Outer outer = new Outer();
        System.out.println(inner.getValue() + inner.getData() + outer.data);
    }
}
```

A. 25

B. Compilation error

C. 20

D. Runtime error

Answer: B

Explanation: Reference type is `Anonymous`, which only declares `getValue()`. `inner.getData()` is not accessible.

9. Non-Static Inner Class Code Syntax

What is the output?

```
public class Outer {
    private int data = 10;

    class Inner {
        private int data = 20;
        private int getData() {
            return data;
        }
        public void main(String[] args) {
            Inner inner = new Inner();
            System.out.println(inner.getData());
        }
    }

    private int getData() {
        return data;
    }

    public static void main(String[] args)
        Outer outer = new Outer();
        Outer.Inner inner = outer.new Inner();
        System.out.printf("%d", outer.getData());
        inner.main(args);
    }
}
```

A. 2010

B. 1020

C. Compilation Error

D. None of these

Answer: C

Explanation: As written, `main` is missing `{`, so it is a compilation error.

10. Nested Interface Implementation

What is the output?

```
interface OuterInterface {
    public void InnerMethod();

    public interface InnerInterface {
        public void InnerMethod();
    }
}

public class Outer implements OuterInterface.InnerInterface, OuterInterface {
    public void InnerMethod() {
        System.out.println(100);
    }

    public static void main(String[] args) {
        Outer obj = new Outer();
        obj.InnerMethod();
```

```
    }
}
```

A. 100

B. Compilation Error

C. Runtime Error

D. None of the above

Answer: A

Explanation: Both interfaces require the same method signature. One implementation satisfies both.

11. Thread join

What is the output?

```
public class Test implements Runnable {
    public void run() {
        System.out.printf("%d", 3);
    }

    public static void main(String[] args) throws InterruptedException {
        Thread thread = new Thread(new Test());
        thread.start();
        System.out.printf("%d", 1);
        thread.join();
        System.out.printf("%d", 2);
    }
}
```

A. 123

B. 213 or 231

C. 132 or 312

D. 321

Answer: C

Explanation: 3 may print before or after `1`, but `join()` ensures 2 prints after the thread finishes.

12. Static Nested Class And Instance Field

What is the output?

```
public class Test {
    private static int value = 20;
    public int s = 15;
    public static int temp = 10;

    public static class Nested {
        private void display() {
            System.out.println(temp + s + value);
        }
    }

    public static void main(String args[]) {
        Test.Nested inner = new Test.Nested();
        inner.display();
    }
}
```

A. Compilation error

B. 1020

C. 101520

D. None of the above

Answer: A

Explanation: Static nested class cannot directly access instance variable `s`.

13. Method Overriding With IOException

What is the output?

```
import java.io.*;

public class Test {
    public void display() throws IOException {
        System.out.println("Test");
```

```
    }
}

class Derived extends Test {
    public void display() throws IOException {
        System.out.println("Derived");
    }

    public static void main(String[] args) throws IOException {
        Derived object = new Derived();
        object.display();
    }
}
```

A. Test

B. Derived

C. Compilation error

D. Runtime error

Answer: B

Explanation: `Derived.display()` overrides `Test.display()` and is called on a Derived object.

14. run() vs start()

What is the output?

```
public class Test extends Thread {
    public void run() {
        System.out.printf("Test ");
    }

    public static void main(String[] args) {
        Test test = new Test();
        test.run();
        test.start();
    }
}
```

A. Compilation error

B. Runtime error

C. Test

D. Test Test

Answer: D

Explanation: `run()` executes normally once. `start()` starts a new thread and calls `run()` again.

15. Nested Interface Access Modifier

For the given code, select the correct answer.

```
public interface Test {
    public int calculate();

    protected interface NestedInterface {
        public void nested();
    }
}
```

A. Compile time error due to NestedInterface

B. Compile time error due to access modifier of NestedInterface

C. No Compile time error

D. NestedInterface cannot hold any function declaration

Answer: B

Explanation: `protected` is not valid for a nested interface declared inside an interface.

16. Constructor Declaration

Which of the following are true about constructor declaration?

A. Constructors can be declared final

B. Constructors can be surrounded by try/catch blocks

C. Constructor cannot throw exception

D. Constructors can hold synchronized code

Answer: D

Explanation: Constructors cannot be final, can throw exceptions, and cannot be declared inside try/catch. They can contain synchronized blocks.

17. Basic Inheritance

What is the output?

```
public class Inheritance {
    public static void main(String[] args) {
        Super i = new Super();
        i.show();
    }
}

class Super {
    public void show() {
        System.out.println("Base");
    }
}

class Sub extends Super {
    public void show() {
        System.out.println("Derived");
    }
}
```

A. Base

B. Compilation Error

C. Derived

D. Runtime Error

Answer: A

Explanation: Object created is `new Super()`, so `Super.show()` runs.

18. Static Method Hiding

What is the output?

```
class Parent {
    static void display() {
        System.out.println("Parent's static display()");
    }
}

class Child extends Parent {
    static void display() {
        System.out.println("Child's static display()");
    }
}

public class Main {
    public static void main(String[] args) {
        Parent obj = new Child();
        obj.display();
    }
}
```

Answer: Parent's static display()

Explanation: Static methods are hidden, not overridden. Method call is resolved using reference type `Parent`.

SQL Query - Slack Employees Salary

Find the sum of salaries of employees who were assigned at least one project and completed none of their projects.

```
WITH slack_employees AS (
    SELECT
        e.id,
        e.salary
    FROM employees e
    JOIN projects p
    ON e.id = p.employee_id
    GROUP BY e.id, e.salary
    HAVING COUNT(p.project_id) >= 1
    AND SUM(CASE WHEN p.End_dt IS NOT NULL THEN 1 ELSE 0 END) = 0
)
SELECT SUM(salary) AS total_slack_salary
FROM slack_employees;
```

Pattern: group by employee, keep employees with zero completed projects, then sum their salaries.

Coding Questions

1. Frequency Of Each String

```
import java.util.*;

class Main {
    public static void main(String[] args) {
        String[] arr = {"apple", "banana", "apple", "cherry", "banana", "apple"};

        HashMap<String, Integer> freq = new HashMap<>();

        for (String s : arr) {
            freq.put(s, freq.getOrDefault(s, 0) + 1);
        }

        System.out.println(freq);
    }
}
```

Time: O(n) average

Space: O(k)

2. Valid Parentheses

```
import java.util.*;

class Main {
    static boolean isBalanced(String s) {
        Stack<Character> stack = new Stack<>();

        for (char ch : s.toCharArray()) {
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            } else {
                if (stack.isEmpty()) return false;

                char top = stack.pop();

                if (ch == ')' && top != '(') return false;
                if (ch == '}' && top != '{') return false;
                if (ch == ']' && top != '[') return false;
            }
        }

        return stack.isEmpty();
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int t = sc.nextInt();

        while (t-- > 0) {
            String str = sc.next();
            System.out.println(isBalanced(str) ? "Balanced" : "Not Balanced");
        }

        sc.close();
    }
}
```

Time: O(n) per string

Space: O(n)

3. Palindrome Linked List

```
class Solution {
    public boolean isPalindrome(ListNode head) {
        if (head == null || head.next == null) return true;

        ListNode slow = head;
        ListNode fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
        }

        ListNode secondHalf = reverse(slow);
        ListNode firstHalf = head;

        while (secondHalf != null) {
            if (firstHalf.val != secondHalf.val) return false;
            firstHalf = firstHalf.next;
            secondHalf = secondHalf.next;
```

```
            }

            return true;
        }

        private ListNode reverse(ListNode head) {
            ListNode prev = null;
            ListNode curr = head;

            while (curr != null) {
                ListNode nextNode = curr.next;
                curr.next = prev;
                prev = curr;
                curr = nextNode;
            }

            return prev;
        }
    }
```

Time: O(n)

Space: O(1)

4. Spiral Level Order Traversal Of Binary Tree

```
    import java.util.*;

    class Solution {
        public List<List<Integer>> zigzagLevelOrder(TreeNode root) {
            List<List<Integer>> result = new ArrayList<>();
            if (root == null) return result;

            Queue<TreeNode> queue = new LinkedList<>();
            queue.add(root);
            boolean leftToRight = true;

            while (!queue.isEmpty()) {
                int size = queue.size();
                LinkedList<Integer> level = new LinkedList<>();

                for (int i = 0; i < size; i++) {
                    TreeNode node = queue.poll();

                    if (leftToRight) {
                        level.addLast(node.val);
                    } else {
                        level.addFirst(node.val);
                    }

                    if (node.left != null) queue.add(node.left);
                    if (node.right != null) queue.add(node.right);
                }

                result.add(level);
                leftToRight = !leftToRight;
            }

            return result;
        }
    }
```

Time: O(n)

Space: O(n)